SURNAME AND INITIALS	STUDENT NUMBER
NDLOVU PS	222295963
NDARANE N	221733320
XAUKA T	221766989
MASHEGO TE	222263255
NETSHIFHEFHE Z	222609437
TOMPANE R	242249372

This is the Project Proposal for Assessment 1 by group 1 from mainstream.

NAME OF SYSTEM, INTRODUCTION

Smart Dual-Verification Alarm System

A React Native mobile application that forces users to physically verify they are awake by capturing a photo of a specified object and, thirty minutes later, a morning activity. The system uses TensorFlow.js and Google Teachable Machine for real-time AI photo verification, making it impossible to dismiss the alarm without genuinely waking up and completing a morning action.

OBJECTIVE / VISION IDE: Visual Studio Code

Project Objective

This project aims to develop a Smart Dual-Verification Alarm System that solves the universal problem of oversleeping and alarm snoozing. Traditional alarms are trivially dismissed with a single tap, causing users to fall back asleep and miss important commitments. This system enforces wakefulness through two mandatory verification stages:

Stage 1 (Primary Alarm): The user must physically get up, locate a randomly selected object in their home, and photograph it before the alarm can be silenced.

Stage 2 (Secondary Alarm – 30 minutes later): The user must photograph proof of a morning activity (e.g., brushing teeth, eating breakfast, or making the bed) to confirm they have transitioned into their daily routine.

The system integrates TensorFlow.js AI to automatically validate the photos, making photospoofing impossible. An additional Sleep Score feature tracks user compliance and assigns a daily productivity score based on wake-up performance. Potential users include students, working professionals, athletes, and anyone requiring discipline in their morning routine.

IDE Chosen: Visual Studio Code (VS Code)

USERS OF THE SYSTEM

1. User / Alarm Owner

The primary user who sets alarm times, selects verification objects/activities, configures alarm sounds, and views their sleep score and history.

2. System Administrator (Optional)

Manages app configuration, default object/activity libraries, and AI model updates for the verification engine.

3. AI Verification Module

Internal software actor (TensorFlow.js / Teachable Machine) that receives captured images and returns a pass/fail verification result.

4. Notification Service

Triggers push notifications and in-app alerts for primary alarm, secondary alarm, missed alarms, and daily sleep score reports.

FUNCTIONAL REQUIREMENTS AND USE CASE DIAGRAMS

The following table captures all functional requirements for the Smart Dual-Verification Alarm System. Each requirement maps to a use case in the diagram below.

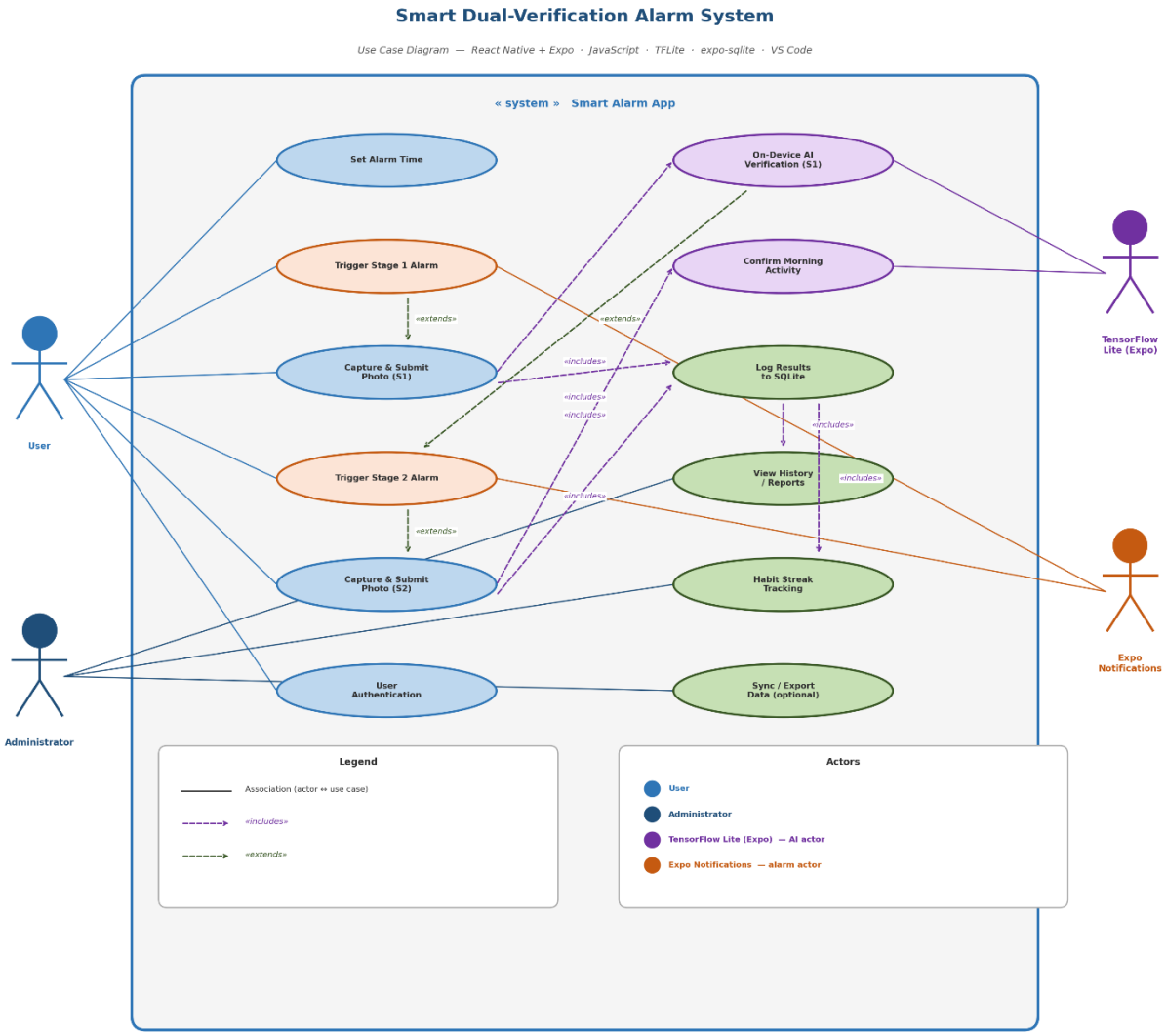
No.	Requirement	Description
Requirement 1 — Set Primary Alarm	**Set Primary Alarm**	User sets a primary alarm time with a customisable sound. System randomly assigns an object from the object library for photo verification.
Requirement 2 — Set Secondary Alarm	**Set Secondary Alarm**	System automatically schedules a secondary alarm 30 minutes after primary dismissal. User can override the interval in settings.
Requirement 3 — Trigger Alarm Sound	**Trigger Alarm Sound**	System plays an intentionally unpleasant, loud alarm sound when alarm time is reached. Sound continues until verification is completed.
Requirement 4 — Capture Object Photo	**Capture Object Photo**	User opens camera within the app and photographs the randomly selected object. System sends image to AI verification module.
Requirement 5 — Capture Activity Photo	**Capture Activity Photo**	User photographs a morning activity (brushing teeth, breakfast, making bed, etc.) for secondary alarm dismissal.
Requirement 6 — AI Photo Verification	**AI Photo Verification**	TensorFlow.js model (trained via Teachable Machine) analyses captured photo and returns a confidence score. Passes if confidence > 75%.
Requirement 7 — Dismiss Alarm	**Dismiss Alarm**	Alarm is dismissed only when AI verification passes. If photo fails verification, alarm restarts and user must try again.
Requirement 8 — Snooze Protection Mode	**Snooze Protection Mode**	Optional feature: system locks snooze completely for users who enable strict mode. Primary alarm cannot be snoozed at all.

Requirement 9 — Sleep Score Engine	**Sleep Score Engine**	System calculates a daily Sleep Score (0–100) based on: alarm set vs actual wake time, number of verification attempts, and time between primary and secondary.
Requirement 10 — Data Logging	**Data Logging (SQLite)**	All alarm events, verification results, photo timestamps and sleep scores are stored locally in SQLite database for history and analytics.
Requirement 11 — Custom	**Custom Object Library**	User can add custom objects to the verification library using the device camera. Objects are stored and randomly selected by the engine.
Object Library		
Requirement 12 — Custom Activity Library	**Custom Activity Library**	User builds a list of accepted morning activities for secondary alarm verification. Activities are labelled and saved.
Requirement 13 — Notification Alerts	**Notification Alerts**	Push notifications alert user 5 minutes before alarm, at alarm time, and again if secondary alarm triggers. Missed alarm triggers escalation alert.
Requirement 14 — Alarm History and Analytics	**Alarm History & Analytics**	Dashboard displays 7-day sleep score trend, verification pass/fail rate, average wake time, and productivity rating.
Requirement 15 — Randomised Object Selection	**Randomised Object Selection**	System randomises which object is required each morning, preventing users from pre-positioning objects near the bed for easy dismissal.
Requirement 16 — Offline Operation	**Offline Operation**	All core features (alarm engine, camera capture, AI verification, SQLite logging) operate fully offline without internet connectivity.

USE CASE DIAGRAM

The diagram below shows all system actors, use cases, and their «include» and «extend» relationships. Tool used: **draw.io (app.diagrams.net)**

USE CASE DIAGRAM — SMART DUAL-VERIFICATION ALARM SYSTEM



ACTORS	USE CASES (SYSTEM BOUNDARY: ALARM VERIFICATION APP)
◆ User / Alarm Owner ◆ System (App) ◆ AI Vision Module ◆ Notification Service	UC01 ««use case»» Set Alarm UC02 ««use case»» Trigger Stage 1 Alarm Sound UC03 ««use case»» Capture Photo (Object) and Submit Photo (Activity) Stage 1 UC04 ««use case»» Trigger Stage 2 Alarm Sound UC05 ««use case»» Capture Photo (Object) and Submit Photo (Activity) Stage 2 UC06 ««use case»» User Authentication UC07 ««use case»» On Device AI Verification UC08 ««use case»» Confirm Morning Activity UC09 ««use case»» Log Results to SQLite UC10 ««use case»» View Alarm History UC11 ««use case»» Habit Streak Tracker UC12 ««use case»» Sync Data

USE CASE RELATIONSHIPS

ID	Use Case	Relationship	Related Use Case
UC03	Capture & Submit Photo (Stage 1)	*«includes»*	UC04 On Device AI Verification (Stage 1)
UC06	Capture & Submit Photo (Stage 2)	*«includes»*	UC07 Confirm Morning Activity

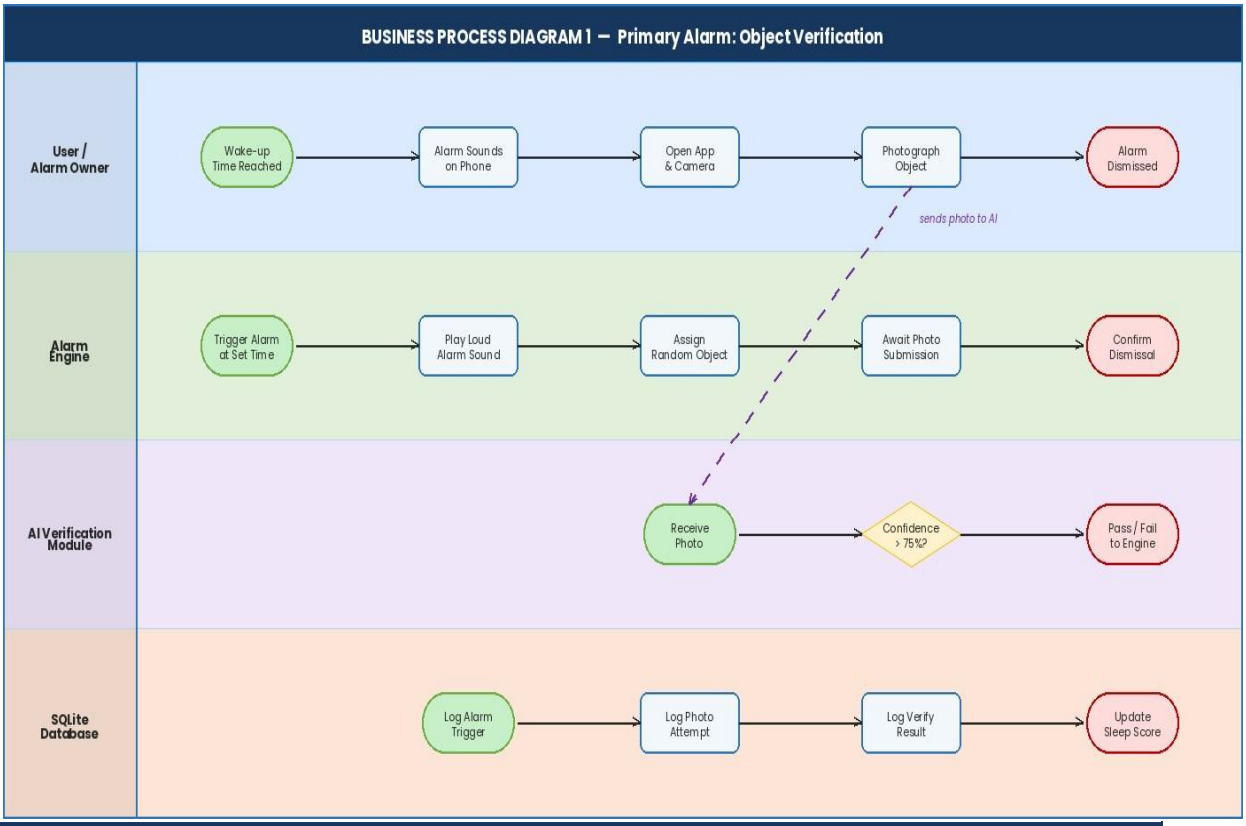
UC03	Capture & Submit Photo (Stage 1)	*«includes»*	UC08 Log Results to SQLite
UC06	Capture & Submit Photo (Stage 2)	*«includes»*	UC08 Log Results to SQLite
UC08	Log Results to SQLite	*«includes»*	UC09 View History
UC08	Log Results to SQLite	*«includes»*	UC11 Habit Streak Tracking
UC02	Trigger Stage 1 Alarm	*«extends»*	UC03 Capture & Submit Photo (Stage 1)
UC04	On Device AI Verification	*«extends»*	UC05 Stage Trigger Stage 2 Alarm
UC05	Trigger Stage 2 Alarm	*«extends»*	UC06 Capture & Submit Photo (Stage 2)

Note: The full graphical UML use case diagram (with actor stick figures, oval use case shapes, include/extend dashed arrows and system boundary rectangle) must be drawn using **https://app.diagrams.net** and inserted above. The table above captures all stereotype relationships for reference.

BUSINESS PROCESS DIAGRAM

The following swimlane diagrams model the two core business processes of the Smart Dual-Verification Alarm System. Diagrams follow BPMN notation conventions (start event ●, task □, decision ◆, end event ◉).

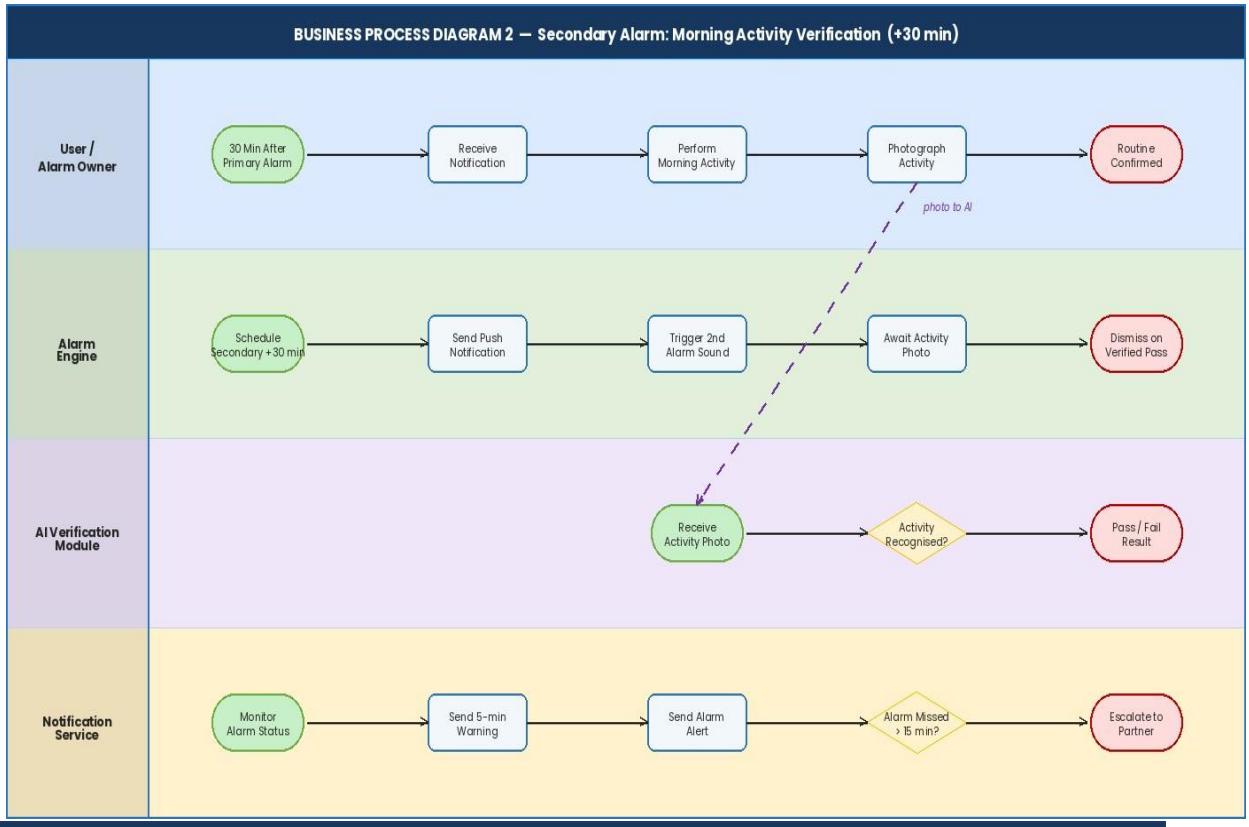
Business Process 1: Primary Alarm Verification



Primary Alarm — Object Verification Process

User	● Wake-up Time Reached → □ Alarm Sounds → □ Open App / Camera → □ Photograph Object → ◉ Alarm Dismissed
Alarm Engine	● Trigger Alarm at Set Time → □ Play Loud Sound → □ Assign Random Object → □ Await Photo Submission → ◉ Confirm Dismissal
AI Module	● Receive Photo → □ Analyse Image →
	◆ Confidence Check → □ Pass / Fail Result → ◉ Send Result to Engine
DB / Log	● Wait for Event → □ Log Alarm Trigger → □ Log Photo Attempt → □ Log Verification Result → ◉ Update Sleep Score

Business Process 2: Secondary Alarm — Activity Verification

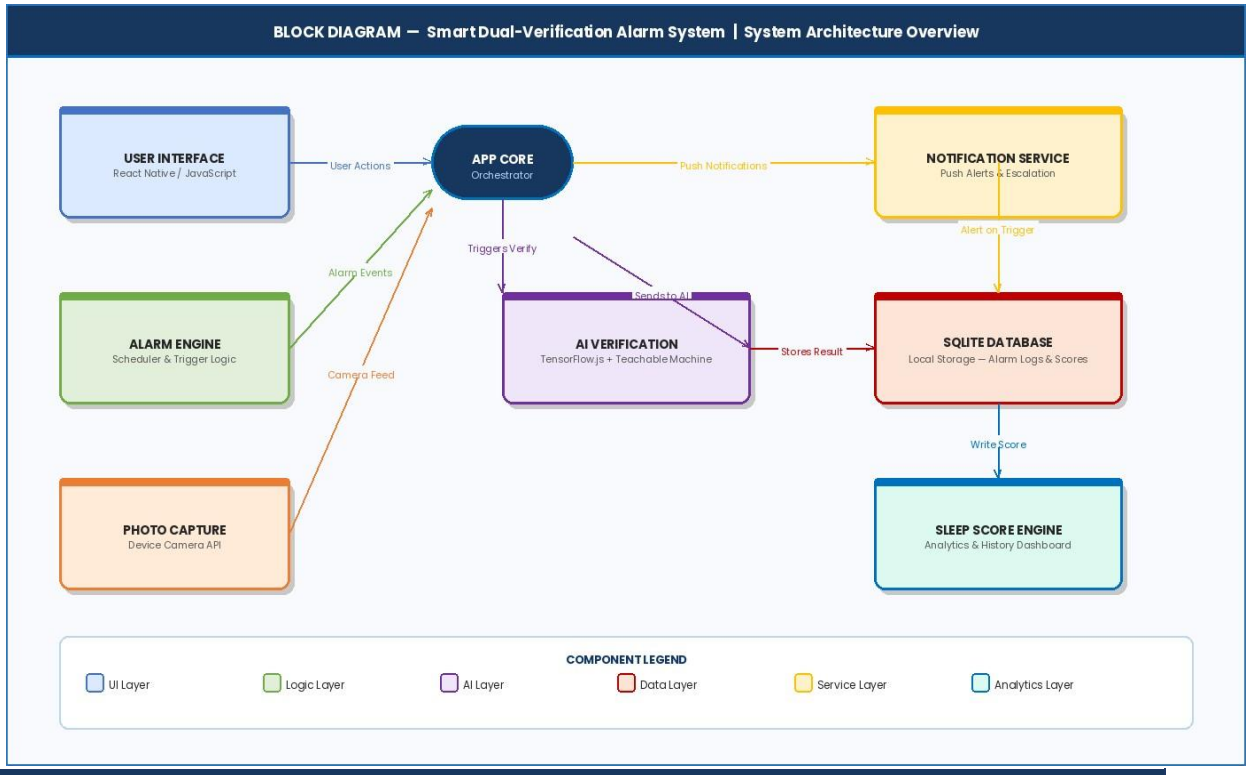


Secondary Alarm — Morning Activity Verification Process (30 min after Primary)

User	● 30 Min After Primary → □ Notified of 2nd Alarm → □ Perform Morning Activity → □ Photograph Activity → ◉ Routine Confirmed
Alarm Engine	● Schedule Secondary (+30 min) → □ Send Push Notification → □ Trigger 2nd Alarm Sound → □ Await Activity Photo → ◉ Dismiss on Pass
AI Module	● Receive Activity Photo → ◆ Check Activity Type → □ Activity Recognised? → □ Pass / Fail Result → ◉ Return to Engine
Notification Service	● Monitor Alarm Status →
	□ Send 5-min Warning → □ Send Alarm Alert → ◆ Send Miss Alert (if ignored) → ◉ Escalate if Needed

BLOCK DIAGRAM

The block diagram below shows the major components of the system and their data flow relationships. All components operate offline using local device resources.



BLOCK DIAGRAM — System Architecture Overview

USER INTERFACE *(React Native / JS)*	**ALARM ENGINE** *(Scheduler & Trigger)*	**PHOTO CAPTURE** *(Device Camera API)*
AI VERIFICATION *(TensorFlow.js / Teachable Machine)*	**DATA LAYER** *(SQLite Local DB)*	**NOTIFICATION** *(Push / Browser Alert)*

Data Flow: User → UI → Alarm Engine → Camera → AI Verification → SQLite DB → Notification Service → User

PLANNING — WORK BREAKDOWN STRUCTURE

The table below serves as the Work Breakdown Structure and Gantt chart plan for the project. Each task has a responsible person, start week, and end week assigned.

No.	Task / Activity	Start	End	Assigned To	Status
1	Project Planning & Requirements	Week 1	Week 1	Group Work	DONE
2	System Design & Proposal (Assig. 1)	Week 2	Week 3	Thato Xauka & Sam	DONE
3	UI Wireframes & Prototype	Week 3	Week 4	Reratilwe & Moneri	IN PROGRESS
4	Alarm Engine Development	Week 4	Week 6	Zoe & Nelisiwe	PENDING
5	Camera & Photo Capture Module	Week 5	Week 7	Thato Xauka	PENDING
6	AI Integration (TensorFlow.js)	Week 6	Week 8	Sam	PENDING
7	SQLite Database Integration	Week 7	Week 8	Nelisiwe	PENDING

8	Notification & Push Alerts	Week 8	Week 9	Reratilwe	**PENDING**
9	Sleep Score & Analytics Module	Week 9	Week 10	Zoe	**PENDING**
10	Testing & Quality Assurance	Week 10	Week 11	Moneri	**PENDING**
11	Documentation & Report	Week 11	Week 12	Sam & Thato Xauka	**PENDING**
12	Final Presentation & Submission	Week 12	Week 12	Group Work	**PENDING**

ADDITIONAL INNOVATIVE FEATURES [BONUS VALUE-ADDS]

The following features distinguish this system from standard alarm apps and competing student projects, ensuring uniqueness and technical depth:

► **Sleep Score Engine:** A proprietary scoring algorithm (0–100) calculates a daily Sleep Score based on: time between alarm set and actual wake time, number of failed photo verification attempts, time gap between primary and secondary alarm completion, and how early/late relative to the set target. Scores are stored in SQLite and displayed as a 7-day trend graph.

► **Randomised Anti-Cheat Object Selection:** Objects for photo verification are randomly selected each morning from a personal library the user builds. The randomisation prevents users from pre-positioning objects near their bed. The app can also require the object to be photographed in a specific room (detected via GPS or room tag).

► **Strict Mode (No Snooze Lock):** A voluntary "Strict Mode" completely disables snooze. Once Strict Mode is active, the alarm cannot be dismissed without completing both verification stages. This mode is designed for users who need extreme accountability.

► **TensorFlow.js AI Confidence Scoring:** Using a model trained via Google Teachable Machine, the AI engine returns a confidence score for each submitted photo. If confidence is below 75%, the photo is rejected and the alarm continues. Users can retrain the model on their own objects for personalised accuracy.

► **Accountability Partner Alerts:** Users can nominate a trusted contact (e.g., parent, friend, study partner). If the primary alarm is not dismissed within 15 minutes, the system automatically sends an SMS/WhatsApp notification to the accountability partner.

► **Morning Routine Streak Tracker:** The system tracks consecutive days of successful dual verification and rewards streaks with in-app achievement badges. This gamification element reinforces habit formation and provides long-term motivation for maintaining discipline.

► **Fully Offline Operation:** Unlike cloud-dependent competitors, the entire verification pipeline (AI model, SQLite database, camera, alarm engine) runs entirely offline on the device. This means the app works on aeroplanes, in areas with no signal, or with mobile data turned off — no privacy concerns about photo uploads.

TECHNICAL STACK SUMMARY

Layer	Technology	Purpose
Frontend	React Native	Cross-platform mobile UI (iOS & Android)
Language	JavaScript	Core application logic and component scripting

AI Engine	TensorFlow.js	Client-side machine learning for photo verification
AI Training	Teachable Machine (Google)	Train custom object/activity recognition models
Editor	Visual Studio Code	Primary IDE for development and debugging
Database	SQLite (local)	Offline-first storage for alarm logs and sleep scores
Diagrams	draw.io / app.diagrams.net	UML use case, business process, and block diagrams
Version Control	Git / GitHub	Source code management and project tracking